# Table of Contents

LAB 03: JavaScript, DOM Manipulation & Fetch API ..... 3
Web Application Development Course ..... 3
1. LAB OVERVIEW ..... 3
Lab Context ..... 3
What Students Will Build ..... 3
Key Learning Shift ..... 4
2. LEARNING OBJECTIVES ..... 4
Technical Skills ..... 4
System Thinking Skills ..... 4
3. THEORY & CONCEPTS (GUIDE) ..... 5
3.1 Core Concepts ..... 5
3.2 The Document Object Model (DOM) ..... 6
3.3 Selecting DOM Elements ..... 7
3.4 Modifying the DOM ..... 7
3.5 Event Handling ..... 8
3.6 Asynchronous JavaScript ..... 10
3.7 Fetch API ..... 11
3.8 JSON (JavaScript Object Notation) ..... 13
3.9 Error Handling Best Practices ..... 14
3.10 Modern JavaScript Features (ES6+) ..... 15
3.11 Common Patterns for Web Applications ..... 17
3.12 Why This Matters in Real-World Systems ..... 18
4. HANDS-ON GUIDE ..... 19
4.1 Setup and File Structure ..... 19
4.2 Step-by-Step: Basic DOM Manipulation ..... 20
4.3 Using Browser DevTools for JavaScript ..... 32
5. EXERCISES ..... 33
Exercise 1 – Basic: Interactive Course Catalog (From Scratch) ..... 33
Exercise 2 – Intermediate: Advanced UI Components ..... 43
Exercise 3 – Advanced: Real-Time Course Availability System ..... 46
6. COMMON MISTAKES ..... 49
Mistake 1: Not Checking if Element Exists ..... 49

Mistake 2: Forgetting to Handle Async Errors ..... 49
Mistake 3: Using innerHTML with User Data ..... 50
Mistake 4: Not Removing Event Listeners ..... 50
Mistake 5: Blocking the Event Loop ..... 51
Mistake 6: Trusting Client-Side Validation ..... 51
Mistake 7: Not Handling Loading States ..... 52
Mistake 8: Mixing Promises and Async/Await ..... 52
Mistake 9: Not Debouncing Search ..... 53
Mistake 10: Global Variables Everywhere ..... 53
7. LAB COMPLETION CHECKLIST ..... 54
JavaScript Fundamentals ..... 54
DOM Manipulation ..... 54
Event Handling ..... 55
Fetch API ..... 55
Asynchronous Code ..... 55
Code Organization ..... 55
User Experience ..... 55
Performance ..... 55
Accessibility ..... 55
Browser Compatibility ..... 56
Security ..... 56
Testing ..... 56
Documentation ..... 56
8. THINKING & DISCUSSION QUESTIONS ..... 56
Question 1: Client vs Server Execution ..... 56
Question 2: Async Execution Flow ..... 57
Question 3: Event Delegation vs Direct Binding ..... 57
Question 4: State Management Challenge ..... 58
Question 5: Race Condition ..... 58
Question 6: Error Handling Strategy ..... 59
Question 7: Performance Optimization ..... 59
Question 8: Memory Leaks ..... 59
Question 9: Optimistic UI ..... 60
Question 10: Real-World Architecture ..... 61

APPENDIX: Additional Resources ..... 61
Official Documentation ..... 61
Learning Resources ..... 61
Tools ..... 61
Practice ..... 62

# LAB 03: JavaScript, DOM Manipulation & Fetch API

## Web Application Development Course

---

## 1. LAB OVERVIEW

**Course:** Web Application Development
**Target Students:** 3rd–4th year Computer Science / IT students
**Prerequisites:** Lab 01 (HTML & HTTP), Lab 02 (CSS)
**Duration:** 3 hours
**Difficulty:** Intermediate → Advanced

### Lab Context

In Labs 01-02, students built static websites. Everything was predetermined - no interactivity, no dynamic content, no response to user actions beyond navigation and form submission. The browser rendered HTML/CSS and that was it.

This lab introduces **client-side programming** with JavaScript. Students will learn to:
- Respond to user interactions (clicks, typing, scrolling)
- Modify page content dynamically (without reload)
- Fetch data from external sources
- Update the UI based on data
- Handle asynchronous operations

**Critical distinction:** This is **client-side** JavaScript. It runs in the browser, not on the server. Understanding this difference is essential before learning server-side programming (Servlets, Spring Boot).

### What Students Will Build

Students will transform their static course catalog into an interactive application:

1. **Dynamic Content Loading**
    - Load course data from JSON files
    - Display courses dynamically (DOM manipulation)
    - Search/filter without page reload

2. **Interactive UI Components**
    - Accordion for course details
    - Tab interface for different course categories
    - Modal dialogs for detailed information
    - Form validation before submission
3. **Asynchronous Data Fetching**
    - Fetch course data using Fetch API
    - Handle loading states
    - Display error messages
    - Retry failed requests

Students will **NOT** use JavaScript frameworks (React, Vue, jQuery). They must understand vanilla JavaScript and the DOM first.

### Key Learning Shift

| Labs 01-02 | Lab 03 |
|---|---|
| Static HTML/CSS | Dynamic content |
| Server determines content | Client determines content |
| Full page reload on interaction | Update parts of page |
| Declarative (HTML/CSS) | Imperative (JavaScript) |
| Synchronous | Asynchronous |

---

## 2. LEARNING OBJECTIVES

By the end of this lab, students will be able to:

### Technical Skills

- Write modern JavaScript (ES6+ features)
- Select and manipulate DOM elements
- Add and remove event listeners
- Create and modify HTML elements dynamically
- Handle asynchronous operations with Promises and async/await
- Use Fetch API to retrieve data
- Parse and display JSON data
- Implement client-side form validation
- Debug JavaScript using browser DevTools

### System Thinking Skills

- Understand the difference between client-side and server-side execution
- Reason about asynchronous vs synchronous code
- Distinguish between what should run on client vs server

- Understand event-driven programming model
- Analyze the request-response cycle with AJAX
- Recognize the limitations of client-side code (security, persistence)
- Plan for error handling and edge cases
- Prepare for backend integration (Labs 04-07)

---

# 3. THEORY & CONCEPTS (GUIDE)

## 3.1 Core Concepts

### *What is JavaScript?*

JavaScript is a **programming language** that runs in the browser (and on servers via Node.js, but that's not our focus). It allows you to:
- React to user events
- Modify page content
- Communicate with servers
- Store data locally
- Calculate and process data

**Key principle:** JavaScript makes web pages **interactive** and **dynamic**.

### *Client-Side vs Server-Side*

```
CLIENT-SIDE (Browser)
┌──────────────────────────────────┐
│ HTML: Structure                  │
│ CSS:  Presentation               │
│ JS:   Behavior & Interactivity   │
│                                  │
│ • Runs in user's browser         │
│ • User can see and modify code   │
│ • Fast (no server round-trip)    │
│ • Limited by browser capabilities│
│ • No direct database access      │
│ • Cannot hide secrets            │
└──────────────────────────────────┘

SERVER-SIDE (Server)
┌──────────────────────────────────┐
│ Java Servlet / Spring Boot       │
│ Database queries                 │
│ Business logic                   │
│ Authentication                   │
│                                  │
│ • Runs on server                 │
│ • User cannot see code           │
│ • Slower (network latency)       │
│ • Full system capabilities       │
│ • Direct database access         │
```

```
  • Can store secrets securely
```

**Critical understanding:**
- Client-side JS is **NOT secure** for sensitive operations
- Users can modify your JavaScript code
- Never trust client-side validation alone
- Client-side is for **UX**, server-side is for **security and business logic**

## 3.2 The Document Object Model (DOM)

When the browser loads HTML, it creates a tree structure called the DOM:

```html
<!DOCTYPE html>
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello</h1>
    <p>World</p>
  </body>
</html>
```

Becomes this DOM tree:

```
document
  └─ html
      ├─ head
      │   └─ title
      │       └─ "My Page"
      └─ body
          ├─ h1
          │   └─ "Hello"
          └─ p
              └─ "World"
```

JavaScript can access and modify this tree:

```javascript
// Get element
const heading = document.querySelector('h1');

// Read content
console.log(heading.textContent); // "Hello"

// Modify content
heading.textContent = 'Hi there!';

// Add class
heading.classList.add('highlight');

// Create new element
```

```
const newParagraph = document.createElement('p');
newParagraph.textContent = 'New paragraph';
document.body.appendChild(newParagraph);
```

**Important:** The DOM is a **live representation**. When JavaScript modifies the DOM, the browser immediately updates what users see.

## 3.3 Selecting DOM Elements

```
// Select single element (returns first match)
const element = document.querySelector('.class-name');
const elementById = document.getElementById('unique-id');
const elementByTag = document.querySelector('p');

// Select multiple elements (returns NodeList)
const elements = document.querySelectorAll('.class-name');
const paragraphs = document.querySelectorAll('p');

// Modern approach: querySelectorAll uses CSS selectors
document.querySelectorAll('.card .title'); // All .title inside .card
document.querySelectorAll('[data-category="web"]'); // Elements with data attribute

// Iterate over NodeList
elements.forEach(element => {
    console.log(element.textContent);
});
```

**Critical:** - `querySelector` returns **first match** or `null` - `querySelectorAll` returns **NodeList** (array-like, but not an array) - Always check if element exists before using it:

```
const element = document.querySelector('.might-not-exist');
if (element) {
    element.textContent = 'Found it!';
} else {
    console.warn('Element not found');
}
```

## 3.4 Modifying the DOM

```
// Get element
const element = document.querySelector('.card');

// Change text content
element.textContent = 'New text';
element.innerHTML = '<strong>Bold text</strong>'; // Careful: security risk with user
data

// Change attributes
element.setAttribute('data-category', 'web');
element.id = 'my-id';
element.className = 'new-class'; // Replaces all classes

// Add/remove/toggle classes (preferred)
```

```
element.classList.add('active');
element.classList.remove('hidden');
element.classList.toggle('expanded'); // Add if absent, remove if present
element.classList.contains('active'); // Check if has class

// Change styles (inline styles, avoid if possible - use classes instead)
element.style.color = 'red';
element.style.backgroundColor = 'blue';
element.style.display = 'none';

// Create new elements
const newDiv = document.createElement('div');
newDiv.textContent = 'Hello';
newDiv.classList.add('card');

// Add to DOM
element.appendChild(newDiv);        // Add as last child
element.prepend(newDiv);            // Add as first child
element.after(newDiv);              // Add after element
element.before(newDiv);             // Add before element

// Remove from DOM
element.remove();                   // Remove element itself
element.removeChild(childElement);  // Remove specific child
```

**Security Warning:**

```
// DANGEROUS - XSS vulnerability
const userInput = '<script>alert("hacked")</script>';
element.innerHTML = userInput; // DON'T DO THIS

// SAFE
element.textContent = userInput; // Escapes HTML
```

## 3.5 Event Handling

Events allow JavaScript to respond to user actions:

```
// Add event listener
const button = document.querySelector('.button');
button.addEventListener('click', function(event) {
    console.log('Button clicked!');
    console.log('Event:', event);
    console.log('Target:', event.target);
});

// Modern syntax with arrow function
button.addEventListener('click', (event) => {
    console.log('Button clicked!');
});

// Remove event listener (must use named function)
```

```
function handleClick(event) {
    console.log('Clicked');
}
button.addEventListener('click', handleClick);
button.removeEventListener('click', handleClick);
```

**Common Events:**

```
// Mouse events
element.addEventListener('click', handler);
element.addEventListener('dblclick', handler);
element.addEventListener('mouseenter', handler);
element.addEventListener('mouseleave', handler);

// Keyboard events
element.addEventListener('keydown', handler);
element.addEventListener('keyup', handler);
element.addEventListener('keypress', handler); // Deprecated

// Form events
form.addEventListener('submit', handler);
input.addEventListener('input', handler);  // Every keystroke
input.addEventListener('change', handler); // After blur
input.addEventListener('focus', handler);
input.addEventListener('blur', handler);

// Document events
document.addEventListener('DOMContentLoaded', handler); // DOM ready
window.addEventListener('load', handler);               // Everything loaded
window.addEventListener('scroll', handler);
window.addEventListener('resize', handler);
```

**Event Object:**

```
element.addEventListener('click', (event) => {
    event.preventDefault();  // Prevent default action (e.g., form submit, link
navigation)
    event.stopPropagation(); // Stop event from bubbling up

    console.log(event.target);      // Element that triggered event
    console.log(event.currentTarget); // Element with listener attached
    console.log(event.type);        // Event type (e.g., 'click')
});
```

**Event Delegation (Important Pattern):**

```
// INEFFICIENT: Add listener to each item
document.querySelectorAll('.item').forEach(item => {
    item.addEventListener('click', handleClick);
});

// EFFICIENT: Add listener to parent, use event delegation
```

```
document.querySelector('.item-list').addEventListener('click', (event) => {
    if (event.target.classList.contains('item')) {
        handleClick(event);
    }
});

// Works even for dynamically added items!
```

## 3.6 Asynchronous JavaScript

JavaScript is **single-threaded**. Long operations would freeze the browser if executed synchronously.

*Synchronous (Blocking)*

```
console.log('Start');
// Hypothetical blocking operation
const data = fetchDataSynchronously(); // Browser freezes here
console.log('Data:', data);
console.log('End');
```

*Asynchronous (Non-Blocking)*

```
console.log('Start');
fetchDataAsynchronously()
    .then(data => {
        console.log('Data:', data);
    });
console.log('End');

// Output:
// Start
// End
// Data: {...}
```

**Three Ways to Handle Async:**

**1. Callbacks (Old way, avoid)**

```
fetchData(function(error, data) {
    if (error) {
        console.error(error);
    } else {
        console.log(data);
    }
});
```

**2. Promises**

```
fetchData()
    .then(data => {
        console.log(data);
        return processData(data);
    })
    .then(processed => {
        console.log(processed);
```

```
    })
    .catch(error => {
        console.error('Error:', error);
    })
    .finally(() => {
        console.log('Cleanup');
    });
```

**3. Async/Await (Modern, preferred)**

```
async function loadData() {
    try {
        const data = await fetchData();
        console.log(data);
        const processed = await processData(data);
        console.log(processed);
    } catch (error) {
        console.error('Error:', error);
    } finally {
        console.log('Cleanup');
    }
}

loadData();
```

**Critical understanding:**
- async function always returns a Promise
- `await` can only be used inside `async` function
- `await` pauses execution until Promise resolves
- Code after `await` runs only after Promise resolves

## 3.7 Fetch API

Fetch API allows making HTTP requests from JavaScript:

```
// Basic GET request
fetch('https://api.example.com/courses')
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        return response.json(); // Parse JSON
    })
    .then(data => {
        console.log('Courses:', data);
    })
    .catch(error => {
        console.error('Fetch error:', error);
    });

// Modern async/await syntax
async function getCourses() {
```

```
    try {
        const response = await fetch('https://api.example.com/courses');

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const data = await response.json();
        console.log('Courses:', data);
        return data;

    } catch (error) {
        console.error('Error fetching courses:', error);
        throw error; // Re-throw to let caller handle
    }
}

// POST request with data
async function createCourse(courseData) {
    try {
        const response = await fetch('https://api.example.com/courses', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify(courseData)
        });

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const result = await response.json();
        return result;

    } catch (error) {
        console.error('Error creating course:', error);
        throw error;
    }
}
```

**Response Methods:**

```
const response = await fetch(url);

response.json()   // Parse JSON
response.text()   // Get as text
response.blob()   // Get as blob (for files)
response.formData() // Get as FormData

// Response properties
```

```
response.ok         // true if status 200-299
response.status     // HTTP status code (200, 404, etc.)
response.statusText // HTTP status message ("OK", "Not Found")
response.headers    // Response headers
```

**Important Gotcha:**

```
// fetch() only rejects on NETWORK errors, not HTTP errors!

// This does NOT throw an error for 404
fetch('https://api.example.com/notfound')
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log('This only catches network errors!'));

// CORRECT: Check response.ok
fetch('https://api.example.com/notfound')
    .then(response => {
        if (!response.ok) {
            throw new Error(`HTTP ${response.status}`);
        }
        return response.json();
    })
    .then(data => console.log(data))
    .catch(error => console.log('Now catches HTTP errors too!'));
```

## 3.8 JSON (JavaScript Object Notation)

JSON is the standard format for data exchange:

```
// JavaScript Object
const course = {
    code: 'CS301',
    title: 'Web Development',
    credits: 3,
    instructor: {
        name: 'Dr. Smith',
        email: 'smith@university.edu'
    },
    prerequisites: ['CS201', 'CS202']
};

// Convert to JSON string
const jsonString = JSON.stringify(course);
console.log(jsonString);
// {"code":"CS301","title":"Web Development","credits":3,...}

// Convert from JSON string to object
const jsonData = '{"code":"CS301","title":"Web Development"}';
const courseObj = JSON.parse(jsonData);
console.log(courseObj.title); // "Web Development"
```

```
// Pretty print JSON
console.log(JSON.stringify(course, null, 2));
```

**Common JSON Patterns:**

```
// Single object
{
    "id": 1,
    "name": "Course Name"
}

// Array of objects
[
    {"id": 1, "name": "Course 1"},
    {"id": 2, "name": "Course 2"}
]

// Nested structure
{
    "course": {
        "code": "CS301",
        "sections": [
            {"number": 1, "time": "10:00"},
            {"number": 2, "time": "14:00"}
        ]
    }
}
```

## 3.9 Error Handling Best Practices

```
// Pattern 1: Try-catch for async/await
async function loadCourses() {
    let courses = [];
    let error = null;
    let loading = true;

    try {
        const response = await fetch('/api/courses');
        if (!response.ok) {
            throw new Error(`Failed to load courses: ${response.status}`);
        }
        courses = await response.json();
    } catch (err) {
        error = err.message;
        console.error('Error loading courses:', err);
    } finally {
        loading = false;
    }

    return { courses, error, loading };
}
```

```
// Pattern 2: Handle specific error types
async function robustFetch(url) {
    try {
        const response = await fetch(url);

        if (!response.ok) {
            if (response.status === 404) {
                throw new Error('Resource not found');
            } else if (response.status === 500) {
                throw new Error('Server error');
            } else {
                throw new Error(`HTTP error: ${response.status}`);
            }
        }

        return await response.json();

    } catch (error) {
        if (error instanceof TypeError) {
            // Network error (no internet, CORS, etc.)
            throw new Error('Network error - check your connection');
        } else {
            // Re-throw other errors
            throw error;
        }
    }
}

// Pattern 3: Retry logic
async function fetchWithRetry(url, retries = 3) {
    for (let i = 0; i < retries; i++) {
        try {
            const response = await fetch(url);
            if (!response.ok) throw new Error(`HTTP ${response.status}`);
            return await response.json();
        } catch (error) {
            if (i === retries - 1) throw error; // Last retry, give up
            console.log(`Retry ${i + 1}/${retries}...`);
            await new Promise(resolve => setTimeout(resolve, 1000)); // Wait 1 second
        }
    }
}
```

## 3.10 Modern JavaScript Features (ES6+)

Students should use modern JavaScript syntax:

```
// 1. let/const (not var)
const name = 'John';  // Cannot reassign
let age = 25;         // Can reassign
age = 26;             // OK
```

```
// 2. Arrow functions
const add = (a, b) => a + b;
const square = x => x * x;
const greet = name => {
    console.log(`Hello ${name}`);
};

// 3. Template literals
const name = 'John';
const age = 25;
const message = `${name} is ${age} years old`;

// 4. Destructuring
const course = { code: 'CS301', title: 'Web Dev', credits: 3 };
const { code, title } = course;

const numbers = [1, 2, 3, 4, 5];
const [first, second, ...rest] = numbers;

// 5. Spread operator
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }

// 6. Default parameters
function greet(name = 'Guest') {
    console.log(`Hello ${name}`);
}

// 7. Array methods
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(n => n * 2);        // [2, 4, 6, 8, 10]
const evens = numbers.filter(n => n % 2 === 0); // [2, 4]
const sum = numbers.reduce((acc, n) => acc + n, 0); // 15
numbers.forEach(n => console.log(n));

// 8. Optional chaining
const user = { profile: { email: 'test@example.com' } };
console.log(user?.profile?.email);  // 'test@example.com'
console.log(user?.address?.city);   // undefined (no error)

// 9. Nullish coalescing
const value = null;
const result = value ?? 'default'; // 'default'

const value2 = 0;
const result2 = value2 ?? 'default'; // 0 (not 'default')
```

## 3.11 Common Patterns for Web Applications

*Loading State Pattern*

```
async function displayCourses() {
    const container = document.querySelector('#course-list');

    // Show loading
    container.innerHTML = '<div class="loading">Loading courses...</div>';

    try {
        const courses = await fetchCourses();

        // Clear loading, show data
        container.innerHTML = '';
        courses.forEach(course => {
            const card = createCourseCard(course);
            container.appendChild(card);
        });

    } catch (error) {
        // Show error
        container.innerHTML = `
            <div class="error">
                Failed to load courses: ${error.message}
                <button onclick="displayCourses()">Retry</button>
            </div>
        `;
    }
}
```

*Search/Filter Pattern*

```
let allCourses = [];

async function initialize() {
    allCourses = await fetchCourses();
    displayCourses(allCourses);
}

function filterCourses(searchTerm) {
    const filtered = allCourses.filter(course => {
        const searchLower = searchTerm.toLowerCase();
        return course.code.toLowerCase().includes(searchLower) ||
               course.title.toLowerCase().includes(searchLower);
    });
    displayCourses(filtered);
}

// Event listener
document.querySelector('#search').addEventListener('input', (e) => {
    filterCourses(e.target.value);
});
```

*Form Validation Pattern*

```
function validateForm(formData) {
    const errors = {};

    if (!formData.name || formData.name.trim() === '') {
        errors.name = 'Name is required';
    }

    if (!formData.email || !formData.email.includes('@')) {
        errors.email = 'Valid email is required';
    }

    if (!formData.course) {
        errors.course = 'Please select a course';
    }

    return errors;
}

form.addEventListener('submit', async (e) => {
    e.preventDefault();

    const formData = new FormData(e.target);
    const data = Object.fromEntries(formData);

    const errors = validateForm(data);

    if (Object.keys(errors).length > 0) {
        displayErrors(errors);
        return;
    }

    try {
        await submitForm(data);
        showSuccess('Form submitted successfully!');
    } catch (error) {
        showError(error.message);
    }
});
```

## 3.12 Why This Matters in Real-World Systems

1. **User Experience:** Modern web apps must feel responsive. Users expect instant feedback, not page reloads.
2. **Performance:** Loading only necessary data (not entire pages) reduces bandwidth and improves speed.
3. **Single Page Applications (SPAs):** Many modern apps (Gmail, Facebook, Twitter) are built on these principles.

4. **API Integration:** Every modern frontend interacts with backend APIs using Fetch or similar technologies.
5. **Career:** JavaScript is essential for frontend development. Every framework (React, Vue, Angular) builds on these fundamentals.
6. **Preparation:** This lab prepares students for:
    - Lab 04-05: Building Java Servlets that respond to these Fetch requests
    - Lab 06-07: Building REST APIs with Spring Boot
    - Lab 08: Full-stack integration

---

# 4. HANDS-ON GUIDE

## 4.1 Setup and File Structure

**Recommended structure:**

```
my-site/
├── index.html
├── courses.html
├── css/
│   └── styles.css
├── js/
│   ├── main.js
│   ├── courses.js
│   └── utils.js
└── data/
    └── courses.json
```

**Link JavaScript in HTML (at end of body):**

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Course Catalog</title>
    <link rel="stylesheet" href="css/styles.css">
</head>
<body>
    <!-- Content -->

    <!-- JavaScript at end of body -->
    <script src="js/utils.js"></script>
    <script src="js/courses.js"></script>
    <script src="js/main.js"></script>
</body>
</html>
```

**Why at end of body?** - Ensures DOM is loaded before JavaScript runs - Improves perceived performance (content shows first) - Alternative: Use `defer` attribute in `<head>`

```
<head>
    <script src="js/main.js" defer></script>
</head>
```

## 4.2 Step-by-Step: Basic DOM Manipulation

*Step 1: Create Sample Data*

Create `data/courses.json`:

```
[
    {
        "code": "CS301",
        "title": "Web Application Development",
        "credits": 3,
        "instructor": "Dr. Smith",
        "schedule": "Mon/Wed 10:00-11:30",
        "description": "Comprehensive introduction to building modern web applications.",
        "prerequisites": ["CS201"],
        "seats": 15,
        "category": "web"
    },
    {
        "code": "CS302",
        "title": "Database Systems",
        "credits": 3,
        "instructor": "Prof. Johnson",
        "schedule": "Tue/Thu 14:00-15:30",
        "description": "Design and implementation of database systems.",
        "prerequisites": ["CS201"],
        "seats": 20,
        "category": "database"
    },
    {
        "code": "CS401",
        "title": "Advanced Web Technologies",
        "credits": 3,
        "instructor": "Dr. Williams",
        "schedule": "Mon/Wed 14:00-15:30",
        "description": "Advanced topics including real-time systems and microservices.",
        "prerequisites": ["CS301"],
        "seats": 10,
        "category": "web"
    },
    {
        "code": "CS303",
        "title": "Computer Networks",
        "credits": 3,
        "instructor": "Dr. Brown",
        "schedule": "Tue/Thu 10:00-11:30",
```

```
        "description": "Fundamentals of computer networking and protocols.",
        "prerequisites": ["CS201"],
        "seats": 25,
        "category": "systems"
    },
    {
        "code": "CS304",
        "title": "Operating Systems",
        "credits": 3,
        "instructor": "Prof. Davis",
        "schedule": "Mon/Wed/Fri 09:00-10:00",
        "description": "Process management, memory management, file systems.",
        "prerequisites": ["CS201"],
        "seats": 18,
        "category": "systems"
    }
]
```

*Step 2: Basic DOM Selection and Manipulation*

Create `js/main.js`:

```
// Wait for DOM to be fully loaded
document.addEventListener('DOMContentLoaded', function() {
    console.log('DOM loaded and ready!');

    // Select elements
    const header = document.querySelector('.site-header h1');
    console.log('Header text:', header.textContent);

    // Modify element
    const changeButton = document.querySelector('#change-title');
    if (changeButton) {
        changeButton.addEventListener('click', function() {
            header.textContent = 'Welcome to Course Catalog!';
            header.classList.add('highlight');
        });
    }

    // Add element
    const addButton = document.querySelector('#add-message');
    if (addButton) {
        addButton.addEventListener('click', function() {
            const message = document.createElement('p');
            message.textContent = 'This message was added with JavaScript!';
            message.classList.add('alert', 'alert-success');

            const container = document.querySelector('.site-main .container');
            container.appendChild(message);
        });
    }
});
```

**Test HTML for this:**

```
<div class="container">
    <button id="change-title" class="button">Change Title</button>
    <button id="add-message" class="button">Add Message</button>
</div>
```

**What to observe:**

1. Open browser console (F12 → Console)
2. Click “Change Title” → See header change
3. Click “Add Message” → See new paragraph appear
4. Inspect Elements tab → See DOM changes

*Step 3: Loading Data with Fetch*

Create `js/courses.js`:

```
/**
 * Fetch courses from JSON file
 * @returns {Promise<Array>} Array of course objects
 */
async function fetchCourses() {
    try {
        const response = await fetch('data/courses.json');

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        const courses = await response.json();
        console.log('Loaded courses:', courses);
        return courses;

    } catch (error) {
        console.error('Error fetching courses:', error);
        throw error;
    }
}

/**
 * Create HTML for a single course card
 * @param {Object} course - Course object
 * @returns {HTMLElement} Course card element
 */
function createCourseCard(course) {
    // Create card container
    const card = document.createElement('div');
    card.classList.add('card', 'course-card');
    card.setAttribute('data-category', course.category);
    card.setAttribute('data-code', course.code);
```

```
    // Create card content
    card.innerHTML = `
        <h3 class="card__title">${course.code} - ${course.title}</h3>
        <div class="card__meta">
            <span><strong>Instructor:</strong> ${course.instructor}</span> |
            <span><strong>Schedule:</strong> ${course.schedule}</span> |
            <span><strong>Credits:</strong> ${course.credits}</span>
        </div>
        <div class="card__content">
            <p>${course.description}</p>
        </div>
        <div class="card__footer">
            <span class="seats ${course.seats < 10 ? 'seats--low' : ''}">
                ${course.seats} seats available
            </span>
            <button class="button button--primary" data-course="${course.code}">
                View Details
            </button>
        </div>
    `;

    return card;
}

/**
 * Display courses in the DOM
 * @param {Array} courses - Array of course objects
 */
function displayCourses(courses) {
    const container = document.querySelector('#course-list');

    if (!container) {
        console.error('Course list container not found');
        return;
    }

    // Clear existing content
    container.innerHTML = '';

    if (courses.length === 0) {
        container.innerHTML = '<p class="empty-state">No courses found</p>';
        return;
    }

    // Create and append cards
    courses.forEach(course => {
        const card = createCourseCard(course);
        container.appendChild(card);
    });

    console.log(`Displayed ${courses.length} courses`);
```

```
}

/**
 * Initialize course page
 */
async function initCoursePage() {
    const container = document.querySelector('#course-list');

    if (!container) {
        return; // Not on courses page
    }

    // Show loading state
    container.innerHTML = '<div class="loading">Loading courses...</div>';

    try {
        const courses = await fetchCourses();
        displayCourses(courses);
    } catch (error) {
        container.innerHTML = `
            <div class="error">
                <p>Failed to load courses: ${error.message}</p>
                <button class="button" onclick="initCoursePage()">Retry</button>
            </div>
        `;
    }
}

// Auto-initialize when DOM is ready
document.addEventListener('DOMContentLoaded', initCoursePage);
```

**HTML for course page:**

```
<main class="site-main">
    <div class="container">
        <h2>Available Courses</h2>
        <div id="course-list" class="card-grid">
            <!-- Courses will be loaded here by JavaScript -->
        </div>
    </div>
</main>
```

**What to observe:**

1. Open DevTools → Network tab
2. Refresh page
3. See request to `courses.json`
4. See courses appear on page
5. Inspect Elements → See dynamically created cards

*Step 4: Adding Search Functionality*

Add to `courses.html`:

```
<div class="search-container">
    <input
        type="text"
        id="course-search"
        class="form-input"
        placeholder="Search courses by code or title..."
    >
</div>
<div id="course-list" class="card-grid"></div>
```

Add to `js/courses.js`:

```
// Store all courses globally for filtering
let allCourses = [];

/**
 * Filter courses based on search term
 * @param {string} searchTerm - Search query
 * @returns {Array} Filtered courses
 */
function filterCourses(searchTerm) {
    if (!searchTerm || searchTerm.trim() === '') {
        return allCourses;
    }

    const search = searchTerm.toLowerCase().trim();

    return allCourses.filter(course => {
        return course.code.toLowerCase().includes(search) ||
               course.title.toLowerCase().includes(search) ||
               course.instructor.toLowerCase().includes(search) ||
               course.description.toLowerCase().includes(search);
    });
}

/**
 * Handle search input
 * @param {Event} event - Input event
 */
function handleSearch(event) {
    const searchTerm = event.target.value;
    const filtered = filterCourses(searchTerm);
    displayCourses(filtered);

    // Update results count
    const resultsCount = document.querySelector('#results-count');
    if (resultsCount) {
        resultsCount.textContent = `${filtered.length} course(s) found`;
```

```
    }
}

/**
 * Initialize course page (updated)
 */
async function initCoursePage() {
    const container = document.querySelector('#course-list');

    if (!container) {
        return;
    }

    // Show loading state
    container.innerHTML = '<div class="loading">Loading courses...</div>';

    try {
        allCourses = await fetchCourses();
        displayCourses(allCourses);

        // Set up search listener
        const searchInput = document.querySelector('#course-search');
        if (searchInput) {
            searchInput.addEventListener('input', handleSearch);
        }

    } catch (error) {
        container.innerHTML = `
            <div class="error">
                <p>Failed to load courses: ${error.message}</p>
                <button class="button" onclick="initCoursePage()">Retry</button>
            </div>
        `;
    }
}
```

**What to observe:** 1. Type in search box → Courses filter in real-time 2. Open Console → See no page reload, no HTTP requests 3. This is **client-side filtering** (fast but only filters loaded data)

*Step 5: Category Filter*

Add to `courses.html`:

```
<div class="filters">
    <button class="filter-btn active" data-category="all">All</button>
    <button class="filter-btn" data-category="web">Web</button>
    <button class="filter-btn" data-category="database">Database</button>
    <button class="filter-btn" data-category="systems">Systems</button>
</div>
```

Add to `js/courses.js`:

```
/**
 * Filter courses by category
 * @param {string} category - Category to filter by
 */
function filterByCategory(category) {
    const filtered = category === 'all'
        ? allCourses
        : allCourses.filter(course => course.category === category);

    displayCourses(filtered);
}

/**
 * Set up category filters
 */
function setupCategoryFilters() {
    const filterButtons = document.querySelectorAll('.filter-btn');

    filterButtons.forEach(button => {
        button.addEventListener('click', function() {
            // Remove active class from all buttons
            filterButtons.forEach(btn => btn.classList.remove('active'));

            // Add active class to clicked button
            this.classList.add('active');

            // Filter courses
            const category = this.getAttribute('data-category');
            filterByCategory(category);
        });
    });
}

// Update initCoursePage to include category filters
async function initCoursePage() {
    const container = document.querySelector('#course-list');

    if (!container) {
        return;
    }

    container.innerHTML = '<div class="loading">Loading courses...</div>';

    try {
        allCourses = await fetchCourses();
        displayCourses(allCourses);

        // Set up search
        const searchInput = document.querySelector('#course-search');
        if (searchInput) {
            searchInput.addEventListener('input', handleSearch);
```

```
        }

        // Set up category filters
        setupCategoryFilters();

    } catch (error) {
        container.innerHTML = `
            <div class="error">
                <p>Failed to load courses: ${error.message}</p>
                <button class="button" onclick="initCoursePage()">Retry</button>
            </div>
        `;
    }
}
```

*Step 6: Modal Dialog for Course Details*

Add to `courses.html`:

```
<!-- Modal overlay -->
<div id="course-modal" class="modal" style="display: none;">
    <div class="modal__overlay"></div>
    <div class="modal__content">
        <button class="modal__close">&times;</button>
        <div id="modal-body">
            <!-- Content loaded dynamically -->
        </div>
    </div>
</div>
```

Add to `css/styles.css`:

```
/* Modal styles */
.modal {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    z-index: 1000;
}

.modal__overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.5);
}

.modal__content {
```

```
    position: relative;
    max-width: 600px;
    margin: 50px auto;
    background: white;
    border-radius: 8px;
    padding: 2rem;
    max-height: 80vh;
    overflow-y: auto;
}

.modal__close {
    position: absolute;
    top: 1rem;
    right: 1rem;
    background: none;
    border: none;
    font-size: 2rem;
    cursor: pointer;
    color: #666;
}

.modal__close:hover {
    color: #000;
}
```

Add to `js/courses.js`:

```
/**
 * Show course details in modal
 * @param {string} courseCode - Course code to display
 */
function showCourseDetails(courseCode) {
    const course = allCourses.find(c => c.code === courseCode);

    if (!course) {
        console.error(`Course ${courseCode} not found`);
        return;
    }

    const modal = document.querySelector('#course-modal');
    const modalBody = document.querySelector('#modal-body');

    // Create modal content
    modalBody.innerHTML = `
        <h2>${course.code} - ${course.title}</h2>
        <div class="course-details">
            <p><strong>Instructor:</strong> ${course.instructor}</p>
            <p><strong>Schedule:</strong> ${course.schedule}</p>
            <p><strong>Credits:</strong> ${course.credits}</p>
            <p><strong>Available Seats:</strong> ${course.seats}</p>

            <h3>Description</h3>
```

```
            <p>${course.description}</p>

            <h3>Prerequisites</h3>
            <ul>
                ${course.prerequisites.map(pre => `<li>${pre}</li>`).join('')}
            </ul>

            <button class="button button--primary"
onclick="enrollInCourse('${course.code}')">
                Enroll Now
            </button>
        </div>
    `;

    // Show modal
    modal.style.display = 'block';
    document.body.style.overflow = 'hidden'; // Prevent background scrolling
}

/**
 * Hide modal
 */
function hideModal() {
    const modal = document.querySelector('#course-modal');
    modal.style.display = 'none';
    document.body.style.overflow = ''; // Restore scrolling
}

/**
 * Set up modal close handlers
 */
function setupModal() {
    const modal = document.querySelector('#course-modal');
    const closeBtn = document.querySelector('.modal__close');
    const overlay = document.querySelector('.modal__overlay');

    // Close on X button
    if (closeBtn) {
        closeBtn.addEventListener('click', hideModal);
    }

    // Close on overlay click
    if (overlay) {
        overlay.addEventListener('click', hideModal);
    }

    // Close on Escape key
    document.addEventListener('keydown', (e) => {
        if (e.key === 'Escape' && modal.style.display === 'block') {
            hideModal();
        }
```

```
    });
}

/**
 * Handle "View Details" button clicks using event delegation
 */
function setupCourseCardListeners() {
    const courseList = document.querySelector('#course-list');

    if (!courseList) return;

    courseList.addEventListener('click', (event) => {
        // Check if clicked element is a "View Details" button
        const button = event.target.closest('button[data-course]');
        if (button) {
            const courseCode = button.getAttribute('data-course');
            showCourseDetails(courseCode);
        }
    });
}

// Update initCoursePage
async function initCoursePage() {
    const container = document.querySelector('#course-list');

    if (!container) {
        return;
    }

    container.innerHTML = '<div class="loading">Loading courses...</div>';

    try {
        allCourses = await fetchCourses();
        displayCourses(allCourses);

        // Set up all event listeners
        const searchInput = document.querySelector('#course-search');
        if (searchInput) {
            searchInput.addEventListener('input', handleSearch);
        }

        setupCategoryFilters();
        setupModal();
        setupCourseCardListeners();

    } catch (error) {
        container.innerHTML = `
            <div class="error">
                <p>Failed to load courses: ${error.message}</p>
                <button class="button" onclick="initCoursePage()">Retry</button>
            </div>
```

```
        `;
    }
}
```

## 4.3 Using Browser DevTools for JavaScript

*Console Tab*

**Essential for debugging:**

```
console.log('Simple message');
console.log('Variable:', myVariable);
console.log('Multiple:', var1, var2, var3);

console.error('Error message');
console.warn('Warning message');
console.info('Info message');

console.table(arrayOfObjects); // Display array as table

console.group('Group Name');
console.log('Message 1');
console.log('Message 2');
console.groupEnd();
```

*Sources/Debugger Tab*

1. Open DevTools → Sources
2. Find your JavaScript file
3. Click line number to add breakpoint
4. Refresh page
5. Code pauses at breakpoint
6. Inspect variables, step through code

**Keyboard shortcuts:** - F8: Resume - F10: Step over - F11: Step into - Shift+F11: Step out

*Network Tab*

Monitor Fetch requests:

1. Open Network tab
2. Filter by “Fetch/XHR”
3. Trigger request
4. Click request to see: - Headers - Response - Timing

*Application Tab*

View local storage, session storage, cookies:

```
// Set/get localStorage
localStorage.setItem('key', 'value');
const value = localStorage.getItem('key');
```

```
// View in DevTools → Application → Local Storage
```

---

# 5. EXERCISES

## Exercise 1 – Basic: Interactive Course Catalog (From Scratch)

**Difficulty:** Beginner → Intermediate
**Estimated Time:** 90 minutes

*Task Description*

Build a complete interactive course catalog application from scratch using HTML, CSS, and JavaScript. This is a standalone single-page application.

*Requirements*

**1. HTML Structure (15-20 minutes)**

Create `index.html` with basic layout:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Course Catalog</title>
    <link rel="stylesheet" href="css/styles.css">
</head>
<body>
    <header class="header">
        <h1>University Course Catalog</h1>
    </header>

    <main class="main">
        <div class="container">
            <!-- Search and filters section -->
            <div class="controls">
                <input
                    type="text"
                    id="search-input"
                    placeholder="Search courses..."
                    class="search-input"
                >

                <div class="filters">
                    <button class="filter-btn active" data-category="all">All</button>
                    <button class="filter-btn" data-category="web">Web</button>
                    <button class="filter-btn" data-category="database">Database</button>
                    <button class="filter-btn" data-category="systems">Systems</button>
```

```
                </div>
            </div>

            <!-- Results count -->
            <div id="results-info" class="results-info"></div>

            <!-- Course list (populated by JavaScript) -->
            <div id="course-list" class="course-grid">
                <!-- Courses will be inserted here -->
            </div>
        </div>
    </main>

    <script src="js/app.js"></script>
</body>
</html>
```

**2. Basic CSS Styling (15-20 minutes)**

Create `css/styles.css` with minimal styling:

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Arial, sans-serif;
    line-height: 1.6;
    color: #333;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 1rem;
}

.header {
    background: #0066cc;
    color: white;
    padding: 2rem 0;
    margin-bottom: 2rem;
}

.controls {
    margin-bottom: 2rem;
}

.search-input {
```

```
    width: 100%;
    padding: 0.75rem;
    font-size: 1rem;
    border: 1px solid #ddd;
    border-radius: 4px;
    margin-bottom: 1rem;
}

.filters {
    display: flex;
    gap: 0.5rem;
    flex-wrap: wrap;
}

.filter-btn {
    padding: 0.5rem 1rem;
    border: 1px solid #ddd;
    background: white;
    cursor: pointer;
    border-radius: 4px;
}

.filter-btn.active {
    background: #0066cc;
    color: white;
    border-color: #0066cc;
}

.results-info {
    margin-bottom: 1rem;
    color: #666;
}

.course-grid {
    display: grid;
    gap: 1.5rem;
    grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
}

.course-card {
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 1.5rem;
    background: white;
}

.course-card.low-seats {
    border-color: #ff6b35;
    border-width: 2px;
}
```

```css
.course-card h3 {
    margin-bottom: 0.5rem;
    color: #0066cc;
}

.course-meta {
    font-size: 0.9rem;
    color: #666;
    margin-bottom: 1rem;
}

.course-description {
    margin-bottom: 1rem;
}

.course-seats {
    font-weight: bold;
}

.course-seats.low {
    color: #ff6b35;
}

.btn {
    padding: 0.5rem 1rem;
    background: #0066cc;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}

.btn:hover {
    background: #0052a3;
}

.loading {
    text-align: center;
    padding: 3rem;
    color: #666;
}

.error {
    background: #fee;
    border: 1px solid #fcc;
    padding: 1rem;
    border-radius: 4px;
    color: #c33;
}
```

**3. Course Data (5 minutes)**

Create `data/courses.json`:

```
[
    {
        "code": "CS301",
        "title": "Web Application Development",
        "credits": 3,
        "instructor": "Dr. Smith",
        "schedule": "Mon/Wed 10:00-11:30",
        "description": "Comprehensive introduction to building modern web applications.",
        "prerequisites": ["CS201"],
        "seats": 15,
        "category": "web"
    },
    {
        "code": "CS302",
        "title": "Database Systems",
        "credits": 3,
        "instructor": "Prof. Johnson",
        "schedule": "Tue/Thu 14:00-15:30",
        "description": "Design and implementation of database systems.",
        "prerequisites": ["CS201"],
        "seats": 8,
        "category": "database"
    },
    {
        "code": "CS401",
        "title": "Advanced Web Technologies",
        "credits": 3,
        "instructor": "Dr. Williams",
        "schedule": "Mon/Wed 14:00-15:30",
        "description": "Advanced topics including real-time systems and microservices.",
        "prerequisites": ["CS301"],
        "seats": 5,
        "category": "web"
    },
    {
        "code": "CS303",
        "title": "Computer Networks",
        "credits": 3,
        "instructor": "Dr. Brown",
        "schedule": "Tue/Thu 10:00-11:30",
        "description": "Fundamentals of computer networking and protocols.",
        "prerequisites": ["CS201"],
        "seats": 25,
        "category": "systems"
    },
    {
        "code": "CS304",
        "title": "Operating Systems",
        "credits": 3,
        "instructor": "Prof. Davis",
```

```
        "schedule": "Mon/Wed/Fri 09:00-10:00",
        "description": "Process management, memory management, file systems.",
        "prerequisites": ["CS201"],
        "seats": 18,
        "category": "systems"
    },
    {
        "code": "CS305",
        "title": "Advanced Databases",
        "credits": 3,
        "instructor": "Dr. Miller",
        "schedule": "Tue/Thu 16:00-17:30",
        "description": "NoSQL, distributed databases, and data warehousing.",
        "prerequisites": ["CS302"],
        "seats": 12,
        "category": "database"
    },
    {
        "code": "CS402",
        "title": "Cloud Computing",
        "credits": 3,
        "instructor": "Prof. Wilson",
        "schedule": "Mon/Wed 13:00-14:30",
        "description": "Cloud architectures, deployment, and scalability.",
        "prerequisites": ["CS303"],
        "seats": 20,
        "category": "systems"
    },
    {
        "code": "CS403",
        "title": "Full Stack Development",
        "credits": 3,
        "instructor": "Dr. Taylor",
        "schedule": "Tue/Thu 10:00-11:30",
        "description": "End-to-end web application development with modern frameworks.",
        "prerequisites": ["CS301", "CS302"],
        "seats": 7,
        "category": "web"
    }
]
```

**4. JavaScript Implementation (Main Task - 40-50 minutes)**

Create `js/app.js` implementing:

**a) Data Fetching:**

```
let allCourses = [];
let filteredCourses = [];
let currentCategory = 'all';
let searchTerm = '';

// Fetch courses from JSON file
```

```
async function fetchCourses() {
    try {
        const response = await fetch('data/courses.json');
        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }
        const courses = await response.json();
        return courses;
    } catch (error) {
        console.error('Error fetching courses:', error);
        throw error;
    }
}
```

**b) DOM Manipulation:**

```
// Create HTML for a course card
function createCourseCard(course) {
    const card = document.createElement('div');
    card.classList.add('course-card');

    // Add low-seats class if needed
    if (course.seats < 10) {
        card.classList.add('low-seats');
    }

    card.innerHTML = `
        <h3>${course.code} - ${course.title}</h3>
        <div class="course-meta">
            <div><strong>Instructor:</strong> ${course.instructor}</div>
            <div><strong>Schedule:</strong> ${course.schedule}</div>
            <div><strong>Credits:</strong> ${course.credits}</div>
        </div>
        <div class="course-description">
            <p>${course.description}</p>
        </div>
        <div class="course-footer">
            <span class="course-seats ${course.seats < 10 ? 'low' : ''}">
                ${course.seats} seats available
            </span>
            <button class="btn" data-course="${course.code}">View Details</button>
        </div>
    `;

    return card;
}

// Display courses in the DOM
function displayCourses(courses) {
    const container = document.getElementById('course-list');
    const resultsInfo = document.getElementById('results-info');
```

```
    // Clear existing content
    container.innerHTML = '';

    // Update results info
    resultsInfo.textContent = `Showing ${courses.length} course(s)`;

    // Check if empty
    if (courses.length === 0) {
        container.innerHTML = '<p class="empty-state">No courses found</p>';
        return;
    }

    // Create and append cards
    courses.forEach(course => {
        const card = createCourseCard(course);
        container.appendChild(card);
    });
}
```

**c) Search & Filter:**

```
// Filter courses based on search and category
function filterCourses() {
    filteredCourses = allCourses.filter(course => {
        // Category filter
        const categoryMatch = currentCategory === 'all' ||
                              course.category === currentCategory;

        // Search filter
        const searchLower = searchTerm.toLowerCase();
        const searchMatch = searchTerm === '' ||
                            course.code.toLowerCase().includes(searchLower) ||
                            course.title.toLowerCase().includes(searchLower) ||
                            course.instructor.toLowerCase().includes(searchLower);

        return categoryMatch && searchMatch;
    });

    displayCourses(filteredCourses);
}

// Handle search input
function handleSearch(event) {
    searchTerm = event.target.value;
    filterCourses();
}

// Handle category filter
function handleCategoryFilter(event) {
    if (!event.target.classList.contains('filter-btn')) return;
```

```
    // Update active button
    document.querySelectorAll('.filter-btn').forEach(btn => {
        btn.classList.remove('active');
    });
    event.target.classList.add('active');

    // Update current category
    currentCategory = event.target.dataset.category;

    // Filter courses
    filterCourses();
}

// Handle view details button
function handleViewDetails(event) {
    if (!event.target.matches('button[data-course]')) return;

    const courseCode = event.target.dataset.course;
    const course = allCourses.find(c => c.code === courseCode);

    if (course) {
        // For now, just alert (students can enhance this)
        alert(`Course Details:\n\n${course.code} - ${course.title}\n\nInstructor:
${course.instructor}\nSchedule: ${course.schedule}\nCredits: ${course.credits}\nSeats:
${course.seats}\n\nPrerequisites: ${course.prerequisites.join(', ')}\n\nDescription:
${course.description}`);
    }
}
```

**d) Initialization:**

```
// Initialize the application
async function init() {
    const courseList = document.getElementById('course-list');

    // Show loading state
    courseList.innerHTML = '<div class="loading">Loading courses...</div>';

    try {
        // Fetch courses
        allCourses = await fetchCourses();
        filteredCourses = allCourses;

        // Display courses
        displayCourses(filteredCourses);

        // Set up event listeners
        const searchInput = document.getElementById('search-input');
        searchInput.addEventListener('input', handleSearch);

        const filtersContainer = document.querySelector('.filters');
```

```
        filtersContainer.addEventListener('click', handleCategoryFilter);

        const courseListEl = document.getElementById('course-list');
        courseListEl.addEventListener('click', handleViewDetails);

    } catch (error) {
        courseList.innerHTML = `
            <div class="error">
                <p>Failed to load courses: ${error.message}</p>
                <button class="btn" onclick="init()">Retry</button>
            </div>
        `;
    }
}

// Start the app when DOM is ready
document.addEventListener('DOMContentLoaded', init);
```

*Technical Requirements*

- External JavaScript file (not inline)
- Use modern JavaScript (ES6+: const/let, arrow functions, async/await)
- Proper error handling with try-catch
- Use event delegation for course card buttons
- Code organized into functions (not all in one big function)

*Constraints*

- NO jQuery or other libraries
- NO JavaScript frameworks
- NO hardcoded courses in HTML
- Must use Fetch API (not XMLHttpRequest)
- Start from scratch (no existing HTML/CSS from previous labs)

*Hints*

- Create the file structure first (folders: css/, js/, data/)
- Get basic HTML/CSS working before JavaScript
- Start with fetching and displaying courses
- Add search functionality second
- Add category filters last
- Test each feature before moving to next
- Use console.log extensively for debugging
- Open DevTools Network tab to see Fetch requests

*What You Must Explain*

**1. Async vs Sync (150 words)** - What happens if fetchCourses() was synchronous? - Why does JavaScript need async/await? - What would the user experience be without async?

**2. Event Delegation (150 words)** - Why use event delegation for course card buttons? - What's the problem with adding listener to each card? - What happens when you dynamically add new courses?

**3. Client-Side Filtering (150 words)** - Your search filters courses already loaded. What are limitations? - What if you have 10,000 courses? - When would you need server-side search instead?

**4. Error Handling (100 words)** - Screenshot your error state - How did you test it? (Hint: change JSON file path to invalid) - Why is error handling important?

---

## Exercise 2 – Intermediate: Advanced UI Components

**Difficulty:** Intermediate
**Estimated Time:** 120 minutes

### *Task Description*

Build advanced interactive components: modal dialog, accordion, tabs, and form validation.

### *Requirements*

**1. Modal Dialog for Course Details**
- Clicking "View Details" opens modal overlay
- Modal shows complete course information
- Modal can be closed by:
- Clicking X button
- Clicking overlay (outside modal)
- Pressing Escape key
- Background scrolling disabled when modal open
- Smooth fade-in animation (CSS transition)

**2. Accordion for Course Sections** Each course can have multiple sections (different times). Create accordion:
- Section headers always visible
- Clicking header expands/collapses section details
- Only one section open at a time
- Visual indicator (arrow) for expanded/collapsed state

**3. Tabs for Different Views** Create tabbed interface:
- Tab 1: Card view (current)
- Tab 2: Table view (all courses in table)
- Tab 3: List view (compact list)
- Switching tabs doesn't reload data - Active tab highlighted

**4. Advanced Form Validation** Create enrollment form with client
-side validation:

- Student Name (required, min 3 characters)
- Student ID (required, format: 8 digits)

- Email (required, valid email format)
- Course Selection (required, dropdown or autocomplete)
- Phone (optional, but if provided must be valid format)
- Comments (optional, max 500 characters, show character count)

Validation requirements:

- Real-time validation (show errors as user types)
- Visual feedback (red border on invalid, green on valid)
- Error messages below each field
- Disable submit button until form is valid
- Prevent submission if validation fails

**5. Loading States and Animations**

- Skeleton loading (placeholder boxes while loading)
- Smooth transitions when filtering/searching
- Fade in/out for course cards
- Spinner for async operations

*Advanced Challenges*

**Challenge 1: Autocomplete Search** Instead of simple search input, create autocomplete:
- Dropdown appears as user types
- Shows matching courses
- Clicking suggestion selects course
- Arrow keys navigate suggestions
- Enter key selects highlighted suggestion

**Challenge 2: Sort Functionality** Add sorting options:
- Sort by: Course Code, Title, Seats Available, Credits
- Ascending/Descending toggle
- Visual indicator of current sort

**Challenge 3: Favorite Courses**
- Heart icon on each course card
- Clicking toggles favorite status
- Store favorites in localStorage
- Persist across page reloads
- Filter to show only favorites

*Technical Requirements*

- Modular code (separate files for different components)
- No global variables (use module pattern or IIFE)
- Accessibility: keyboard navigation works
- Performance: smooth animations (60fps)
- Memory management: remove event listeners when needed

*Incomplete Requirements (You Decide)*

**1. Modal Accessibility:** How do you make the modal accessible?
- Focus management (where does focus go when modal opens?)
- Screen reader announcements
- Tab key behavior (trap focus in modal?)

**2. Form Submission:** The form validates, but where does data go?
- Do you actually submit to server?
- Do you just log to console?
- Do you show success message? Design this yourself.

**3. Animation Performance:** You have smooth animations. But what if someone:
- Prefers reduced motion?
- Is on a slow device? How do you handle this?

*Hints*

- Build one component at a time
- Test on mobile (touch events differ from mouse)
- Use CSS classes for state (is-open, is-active, is-invalid)
- Store state in data attributes where appropriate
- Use requestAnimationFrame for smooth animations

*What You Must Explain*

**1. Modal Implementation (300 words)** Explain your modal implementation:
- How do you prevent background scrolling?
- How do you handle focus management?
- Why did you use specific event listeners?
- What accessibility considerations did you make?

**2. State Management (250 words)** Your app has state (which tab is active, which courses are favorited, modal open/closed):
- How do you manage state?
- Where is state stored?
- What happens on page reload?
- How would you scale this to more complex state?

**3. Performance Analysis (300 words)** Using DevTools Performance tab:
- Record interaction (opening modal, switching tabs)
- Analyze frame rate
- Identify any janky animations
- What optimizations did you make?
- What would you optimize if needed?

## Exercise 3 – Advanced: Real-Time Course Availability System

**Difficulty:** Advanced
**Estimated Time:** 150 minutes

### *Task Description*

Build a sophisticated course enrollment system with simulated real-time updates, optimistic UI, and complex state management.

### *Requirements*

**1. Simulated Backend API** Create a mock API using JavaScript:
- Simulate network latency (random 500-1500ms delay)
- Random failures (10% chance)
- Simulate concurrent users (seats decrease randomly)
- API methods:

  - `GET /api/courses` - List all courses
  - `GET /api/courses/:code` - Get single course
  - `POST /api/enrollment` - Enroll in course
  - `DELETE /api/enrollment/:id` - Cancel enrollment

```
// Example mock API structure
const mockAPI = {
    async getCourses() {
        await delay(randomDelay());
        if (randomFailure()) throw new Error('Network error');
        return courses;
    },
    async enrollInCourse(courseCode, studentData) {
        await delay(randomDelay());
        if (randomFailure()) throw new Error('Enrollment failed');
        // Decrease available seats
        // Return enrollment confirmation
    }
};
```

**2. Real**
**-Time Updates (Simulated)** Simulate other students enrolling:
- Every 5-10 seconds, random course loses 0-2 seats
- Update UI automatically when this happens
- Show notification “X seats remaining in Y course”
- Use polling or simulate WebSocket

**3. Optimistic UI Updates** When student enrolls:
- Immediately update UI (decrease seats, show “Enrolled”)
- Make API call in background
- If API succeeds: keep optimistic update
- If API fails: revert UI, show error
- Handle edge case: seat taken by someone else

**4. Shopping Cart System** Before enrolling, students add courses to cart:
- Cart stored in memory and localStorage
- Shows total credits
- Detects conflicts (same time slot)
- Shows prerequisites warnings
- "Checkout" enrolls in all courses

**5. Complex Validation**
- Check prerequisites (can't enroll if missing prerequisites)
- Check time conflicts (can't enroll in courses at same time)
- Check credit limit (max 18 credits per semester)
- Check seat availability (can't enroll if full)
- Validate student eligibility (custom rules)

**6. Advanced Features**

**Waitlist System:**
- If course full, option to join waitlist
- Show waitlist position
- Simulate spots opening (random)
- Notify when spot available

**Course Recommendations:**
- Based on enrolled courses, recommend related courses
- Use prerequisites as hints
- Show "Students who took X also took Y"

**Schedule Visualization:**
- Weekly calendar view
- Drag courses onto schedule
- Visual conflict detection
- Export schedule as image/PDF

*Technical Challenges*

**Challenge 1: Concurrent Requests** Handle multiple simultaneous enrollments:
- Queue system for API calls
- Don't let user spam "Enroll" button
- Show pending state
- Handle race conditions

**Challenge 2: Offline Support** What happens when internet disconnects?
- Detect offline state
- Queue actions while offline
- Sync when back online
- Show appropriate UI

**Challenge 3: Undo/Redo** Implement undo/redo for enrollments:
- Command pattern

- History stack
- Keyboard shortcuts (Ctrl+Z, Ctrl+Y)

**Challenge 4: Performance with Large Dataset** What if there are 1000 courses?
- Virtual scrolling
- Lazy loading
- Debounced search
- Measure and optimize

*Edge Cases to Handle*

**1. Double Enrollment Prevention** User clicks "Enroll" twice quickly: - How do you prevent double enrollment? - Disable button immediately? - Check if already enrolled?

**2. Stale Data** User opens page at 9:00 AM, enrolls at 11:00 AM: - Course data is 2 hours old - Seats might be full now - How do you handle this?

**3. Partial Failures** User enrolls in 5 courses, 3 succeed, 2 fail: - What's the user experience? - Do you rollback successful ones? - How do you communicate this?

**4. Race Conditions** Last seat available, two users click "Enroll" simultaneously: - Who gets it? - How do you handle the other user? - What's the error message?

**5. Browser Refresh** User adds courses to cart, refreshes page: - Does cart persist? - What about pending enrollments? - How do you recover state?

*Hints*

- Use Promises for mock API
- Use setTimeout to simulate delays
- Use Math.random() for failures
- Store cart in localStorage
- Use Date objects for schedule conflicts
- Consider using a state management pattern (e.g., reducer pattern)

*What You Must Explain (Comprehensive Report)*

**1. Architecture Documentation (400 words)** Document your system architecture:
- What modules/files did you create?
- How do they communicate?
- What's your data flow?
- How do you manage state?
- Include a diagram (ASCII art is fine)

**2. Mock API Design (300 words)** Explain your mock API:
- Why did you choose specific delay ranges?
- How realistic is your failure simulation?
- What HTTP methods did you simulate?
- How would you replace this with real API?

*Deliverables*

1. **Source Code**
    - Well-commented JavaScript
    - Organized into modules
    - README with setup instructions
2. **Technical Report**
    - Answers to all questions above
    - Architecture diagrams
    - Screenshots of key features
3. **Demo Video** (Optional but recommended)
    - 3-5 minute walkthrough
    - Show key features
    - Demonstrate error handling
    - Show edge cases

---

## 6. COMMON MISTAKES

### Mistake 1: Not Checking if Element Exists

**Wrong:**

```
const button = document.querySelector('.submit-btn');
button.addEventListener('click', handleClick); // Error if button doesn't exist
```

**Right:**

```
const button = document.querySelector('.submit-btn');
if (button) {
    button.addEventListener('click', handleClick);
} else {
    console.warn('Submit button not found');
}
```

**Why it matters:** Your code crashes when element is missing. Always check before using.

---

### Mistake 2: Forgetting to Handle Async Errors

**Wrong:**

```
async function loadCourses() {
    const response = await fetch('/api/courses');
    const courses = await response.json();
    displayCourses(courses);
}
// If fetch fails, user sees nothing, no error message
```

**Right:**

```
async function loadCourses() {
    try {
        const response = await fetch('/api/courses');
        if (!response.ok) {
            throw new Error(`HTTP ${response.status}`);
        }
        const courses = await response.json();
        displayCourses(courses);
    } catch (error) {
        console.error('Failed to load courses:', error);
        showErrorMessage('Failed to load courses. Please try again.');
    }
}
```

---

## Mistake 3: Using innerHTML with User Data

**Wrong:**

```
const searchTerm = searchInput.value;
container.innerHTML = `<p>Results for: ${searchTerm}</p>`;
// XSS vulnerability!
```

If user types: `<script>alert('hacked')</script>`, it executes!

**Right:**

```
const searchTerm = searchInput.value;
const message = document.createElement('p');
message.textContent = `Results for: ${searchTerm}`;
container.appendChild(message);
// textContent escapes HTML
```

---

## Mistake 4: Not Removing Event Listeners

**Wrong:**

```
function createButton() {
    const btn = document.createElement('button');
    btn.addEventListener('click', handleClick);
    container.appendChild(btn);
}

// Call this 100 times
for (let i = 0; i < 100; i++) {
    createButton();
}
// Memory leak: if you remove buttons but keep references to them
```

**Right:**

```
// Use event delegation instead
container.addEventListener('click', (e) => {
    if (e.target.tagName === 'BUTTON') {
        handleClick(e);
    }
});

// Create buttons without individual listeners
for (let i = 0; i < 100; i++) {
    const btn = document.createElement('button');
    container.appendChild(btn);
}
```

---

## Mistake 5: Blocking the Event Loop

**Wrong:**

```
function processLargeArray(arr) {
    for (let i = 0; i < 1000000; i++) {
        // Heavy computation
        arr[i] = arr[i] * 2;
    }
    // Browser freezes during this
}
```

**Right:**

```
async function processLargeArray(arr) {
    const chunkSize = 1000;
    for (let i = 0; i < arr.length; i += chunkSize) {
        const chunk = arr.slice(i, i + chunkSize);
        chunk.forEach((val, idx) => {
            arr[i + idx] = val * 2;
        });
        // Yield to browser
        await new Promise(resolve => setTimeout(resolve, 0));
    }
}
```

---

## Mistake 6: Trusting Client-Side Validation

**Wrong thinking:** “I validated on client, so data is safe.”

**Reality:** - Users can disable JavaScript - Users can modify your code - Users can send HTTP requests directly (bypassing your UI)

**Right approach:**

```
// Client-side validation for UX
function validateForm(data) {
    // Validate here
}

// But ALWAYS validate on server too!
// Server-side (Java Servlet, Spring Boot):
// - Validate again
// - Sanitize inputs
// - Check authentication
// - Check authorization
```

**Why it matters:** Security. Never trust the client.

---

## Mistake 7: Not Handling Loading States

**Wrong:**

```
async function loadCourses() {
    const courses = await fetchCourses();
    displayCourses(courses);
}
// User sees nothing while loading
```

**Right:**

```
async function loadCourses() {
    showLoading();
    try {
        const courses = await fetchCourses();
        displayCourses(courses);
    } catch (error) {
        showError(error);
    } finally {
        hideLoading();
    }
}
```

---

## Mistake 8: Mixing Promises and Async/Await

**Confusing:**

```
async function loadData() {
    const response = await fetch('/api/data');
    return response.json().then(data => {
        return processData(data);
    }).catch(error => {
        console.error(error);
    });
```

```
}
// Mixing styles is confusing
```

**Clear:**

```
async function loadData() {
    try {
        const response = await fetch('/api/data');
        const data = await response.json();
        return processData(data);
    } catch (error) {
        console.error(error);
        throw error;
    }
}
```

---

## Mistake 9: Not Debouncing Search

**Wrong:**

```
searchInput.addEventListener('input', async (e) => {
    const results = await searchCourses(e.target.value);
    displayResults(results);
});
// Makes API call on EVERY keystroke!
```

**Right:**

```
function debounce(func, delay) {
    let timeout;
    return function(...args) {
        clearTimeout(timeout);
        timeout = setTimeout(() => func.apply(this, args), delay);
    };
}

const debouncedSearch = debounce(async (searchTerm) => {
    const results = await searchCourses(searchTerm);
    displayResults(results);
}, 300);

searchInput.addEventListener('input', (e) => {
    debouncedSearch(e.target.value);
});
// Only searches after user stops typing for 300ms
```

---

## Mistake 10: Global Variables Everywhere

**Wrong:**

```
var courses = [];
var filteredCourses = [];
var selectedCourse = null;
var isLoading = false;
// Global namespace pollution
```

**Right:**

```
// Module pattern
const CourseApp = (function() {
    // Private variables
    let courses = [];
    let filteredCourses = [];
    let selectedCourse = null;
    let isLoading = false;

    // Public API
    return {
        init() {
            // Initialize
        },
        loadCourses() {
            // Load courses
        }
    };
})();

// Or use ES6 modules
```

---

## 7. LAB COMPLETION CHECKLIST

### JavaScript Fundamentals

- ☐ Used modern JavaScript (const/let, arrow functions, template literals)
- ☐ No use of var
- ☐ Proper async/await usage
- ☐ Try-catch blocks for error handling
- ☐ Functions are small and focused (single responsibility)

### DOM Manipulation

- ☐ Elements selected correctly (querySelector/querySelectorAll)
- ☐ Checked if elements exist before using
- ☐ Used textContent instead of innerHTML where appropriate
- ☐ Created elements dynamically (createElement, appendChild)
- ☐ Managed classes with classList

## Event Handling

- ☐ Used addEventListener (not onclick attribute)
- ☐ Event delegation for dynamic elements
- ☐ preventDefault() used where needed
- ☐ Event listeners removed when no longer needed
- ☐ Keyboard events handled (Enter key, Escape, etc.)

## Fetch API

- ☐ Used Fetch API (not XMLHttpRequest)
- ☐ Checked response.ok before parsing
- ☐ Proper error handling
- ☐ Loading states shown
- ☐ Error messages displayed to user

## Asynchronous Code

- ☐ Used async/await (not callbacks)
- ☐ Proper Promise handling
- ☐ No race conditions
- ☐ Sequential vs parallel requests used appropriately

## Code Organization

- ☐ External JavaScript files (not inline)
- ☐ Code organized into functions
- ☐ No global variables (or minimized)
- ☐ Comments for complex logic
- ☐ Consistent naming convention

## User Experience

- ☐ Loading indicators shown during async operations
- ☐ Error messages are user-friendly
- ☐ Success feedback provided
- ☐ Form validation provides clear feedback
- ☐ Buttons disabled during processing

## Performance

- ☐ No unnecessary re-renders
- ☐ Debouncing used for search/scroll/resize
- ☐ Event delegation used where appropriate
- ☐ No memory leaks (listeners cleaned up)

## Accessibility

- ☐ Keyboard navigation works

- ☐ Focus management (especially in modals)
- ☐ ARIA attributes where needed
- ☐ Error messages associated with form fields

### Browser Compatibility

- ☐ Tested in Chrome and Firefox
- ☐ No console errors
- ☐ Works with JavaScript enabled
- ☐ Degrades gracefully if Fetch fails

### Security

- ☐ No innerHTML with user data
- ☐ Aware that client-side validation is not secure
- ☐ No sensitive data in client-side code

### Testing

- ☐ Tested happy path (everything works)
- ☐ Tested error scenarios (network failure, invalid data)
- ☐ Tested edge cases (empty data, very long input)
- ☐ Tested on mobile (touch events)

### Documentation

- ☐ README with setup instructions
- ☐ Code comments for complex logic
- ☐ Questions answered thoroughly

---

## 8. THINKING & DISCUSSION QUESTIONS

### Question 1: Client vs Server Execution

**Scenario:** You're building a course enrollment system.

**Which operations should run on client vs server?**

```
Operation A: Validate email format
Operation B: Check if student has prerequisites
Operation C: Highlight search results
Operation D: Deduct seats from database
Operation E: Calculate total credits
Operation F: Authenticate user login
```

**Questions:** a) Which should run on client? Which on server? Which on both?

b) For each operation, explain your reasoning.

c) What happens if you ONLY validate on client?

d) What are the trade-offs of doing more on client vs server?

---

## Question 2: Async Execution Flow

Given this code:

```
console.log('A');

setTimeout(() => {
    console.log('B');
}, 0);

Promise.resolve().then(() => {
    console.log('C');
});

console.log('D');
```

**Questions:** a) What order do these log? Why?

b) Why doesn't setTimeout with 0ms run immediately?

c) What's the difference between setTimeout and Promise?

d) How does the event loop work?

---

## Question 3: Event Delegation vs Direct Binding

**Scenario:** You have 100 course cards, each with a "View Details" button.

**Approach A: Direct Binding**

```
document.querySelectorAll('.view-btn').forEach(btn => {
    btn.addEventListener('click', handleClick);
});
```

**Approach B: Event Delegation**

```
document.querySelector('#course-list').addEventListener('click', (e) => {
    if (e.target.classList.contains('view-btn')) {
        handleClick(e);
    }
});
```

**Questions:** a) What's the memory difference between these approaches?

b) What happens if you add a new course card dynamically?

c) Are there cases where direct binding is better?

d) How do you debug event delegation issues?

---

## Question 4: State Management Challenge

**Scenario:** Your app has this state: - List of courses (from server) - Search term (from user input) - Selected category (from filter buttons) - Shopping cart (courses to enroll in) - User info (name, ID, etc.)

**Questions:** a) Where should each piece of state be stored?

b) What happens to state when user refreshes page?

c) How do you keep UI in sync with state?

d) When would you need a state management library (Redux, MobX)?

e) How would you implement undo/redo?

---

## Question 5: Race Condition

**Scenario:**

```
let selectedCourse = null;

// User clicks course A
setTimeout(() => {
    selectedCourse = 'CS301';
    showDetails('CS301');
}, 1000);

// Immediately clicks course B
setTimeout(() => {
    selectedCourse = 'CS302';
    showDetails('CS302');
}, 500);

// Which course details are shown? Why?
```

**Questions:** a) What's the final value of selectedCourse?

b) What details are shown to the user?

c) How would you fix this race condition?

d) In a real app with API calls, how could this go wrong?

---

## Question 6: Error Handling Strategy

**Scenario:** Your `fetchCourses()` can fail because: - Network is down (TypeError) - Server returns 404 (course data not found) - Server returns 500 (server error) - JSON is malformed (SyntaxError) - Request times out

**Questions:** a) How would you handle each error differently?

b) What should users see for each error?

c) Should you retry? If so, how many times?

d) How do you log errors for debugging?

e) What's the difference between recoverable and unrecoverable errors?

---

## Question 7: Performance Optimization

**Scenario:** You have a search that filters 1000 courses.

**Current implementation:**

```
searchInput.addEventListener('input', (e) => {
    const filtered = allCourses.filter(course => {
        return course.title.toLowerCase().includes(e.target.value.toLowerCase());
    });
    displayCourses(filtered);
});
```

**Questions:** a) What performance issues might this have?

b) How would debouncing help? Show code.

c) What if filtering still takes 200ms? (too long)

d) How would you optimize the filter operation itself?

e) When would you need to move search to server-side?

---

## Question 8: Memory Leaks

**Scenario:**

```
function createCourseCard(course) {
    const card = document.createElement('div');
    const btn = document.createElement('button');

    btn.addEventListener('click', () => {
        showCourseDetails(course);
    });
```

```
    card.appendChild(btn);
    return card;
}

// Called 1000 times
for (let i = 0; i < 1000; i++) {
    const card = createCourseCard(courses[i]);
    container.appendChild(card);
}

// Later, clear all cards
container.innerHTML = '';

// Memory leak? Why or why not?
```

**Questions:** a) Is there a memory leak here?

b) What happens to the event listeners?

c) How would you fix this?

d) When do you actually need to remove listeners manually?

---

## Question 9: Optimistic UI

**Scenario:** User clicks “Enroll” in a course.

**Approach A: Wait for Server**

```
async function enroll(courseCode) {
    showLoading();
    try {
        await api.enrollInCourse(courseCode);
        updateUI(); // Only update after success
        showSuccess();
    } catch (error) {
        showError();
    }
}
```

**Approach B: Optimistic Update**

```
async function enroll(courseCode) {
    updateUI(); // Update immediately
    try {
        await api.enrollInCourse(courseCode);
        showSuccess();
    } catch (error) {
        revertUI(); // Undo changes
        showError();
```

```
    }
}
```

**Questions:** a) What's the user experience difference?

b) What are the risks of optimistic updates?

c) When should you NOT use optimistic UI?

d) How do you handle the "revert" operation?

e) What if multiple operations are in flight?

---

## Question 10: Real-World Architecture

**Scenario:** You're building a large course management system with: - 50+ different pages - Shared components (header, navigation, course cards) - Complex state (user session, shopping cart, notifications) - API communication - Real-time updates

**Questions:** a) How would you organize your JavaScript code?

b) At what point does vanilla JavaScript become too complex?

c) What problems would a framework (React/Vue) solve?

d) What new problems would a framework introduce?

e) How would you decide whether to use a framework?

f) If you were building this for production, what would your tech stack be? Why?

---

# APPENDIX: Additional Resources

## Official Documentation

- MDN JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise

## Learning Resources

- JavaScript.info: https://javascript.info/
- Eloquent JavaScript (free book): https://eloquentjavascript.net/
- You Don't Know JS (free books): https://github.com/getify/You-Dont-Know-JS

## Tools

- ESLint (code linting): https://eslint.org/

- Prettier (code formatting): https://prettier.io/
- Chrome DevTools: https://developer.chrome.com/docs/devtools/

## Practice

- JavaScript30 (30 day challenge): https://javascript30.com/
- freeCodeCamp: https://www.freecodecamp.org/

---

**End of Lab 03**

---

**Notes for Instructor:**

This lab is the critical bridge between static websites and full
-stack applications. Students must master these concepts before learning Servlets and Spring Boot.

Key pedagogical points: 1. JavaScript is CLIENT
-SIDE
- emphasize security implications 2. Async programming is hard
- spend time on it 3. DOM manipulation is foundational for all frameworks 4. Error handling is not optional 5. DevTools is essential for debugging

Expected student struggles:
- Async/await and Promises (callback hell flashbacks)
- “Why doesn’t my code run in order?”
- XSS vulnerabilities with innerHTML
- Race conditions
- Memory leaks from event listeners
- Thinking they can trust client-side code

Teaching tips:
- Live code demonstrations (critical for async)
- Show common bugs and how to debug them
- Emphasize DevTools usage
- Compare client-side JS to server-side Java (preview)
- Discuss real-world architecture decisions

Assessment criteria:
- Code quality and organization: 25%
- Async handling and error management: 25%
- DOM manipulation correctness: 20%
- User experience (loading states, errors): 15%
- Written explanations: 15%

Common pitfalls to watch for:
- Not checking if elements exist
- Using innerHTML with user data

- No error handling
- Blocking the event loop
- Trusting client-side validation
- Memory leaks from listeners

**Connection to future labs:**

- Lab 04 (Servlets): These Fetch requests will hit real endpoints
- Lab 05 (JSP): Server-side rendering vs client-side rendering
- Lab 06-07 (Spring Boot): Building REST APIs that these fetch calls consume
- Lab 08: Full integration of everything

---